



Project Sentinel

Privacy Automation Platform

Project Bible

Project Requirements and Philosophy Specification

DOCUMENT VERSION	0.1
STATUS	Concept Specification
AUTHOR	Jan van Hout
DATE	August 23, 2026

Table of Contents

1	Project Overview	1
1.1	Goals	1
2	Core Philosophy	2
2.1	Local-First Architecture	2
2.2	User-Controlled Automation	2
2.3	Priority Order	2
3	Privacy Principles	4
3.1	Data Minimization	4
3.2	No Central User Database	4
4	Supported Platforms	5
5	Installation	6
6	User Authentication and Encryption	7
6.1	Local Authentication	7
6.2	Password Requirements	7
6.3	Encryption Model	7
6.4	Recovery	8
6.4.1	Layer 1 — The Data Encryption Key (DEK)	8
6.4.2	Layer 2 — Key Slots	8
6.5	How unlocking works	9
6.6	Why This Is Secure	9
6.7	Recovery Key Regeneration	9
6.8	Why This Preserves the Security Guarantee	10
6.9	Opt-Out	10
7	Identity Management	11
7.1	Identity Profiles	11
8	Privacy Rules	12
9	Legal Framework Support	13

10 Privacy Request Types	14
11 Request Workflow	15
12 Bulk Actions	16
13 Company Database	17
13.1 Company Records	17
14 Community Database System	18
14.1 Review Process	18
14.2 Submission Verification	18
14.3 Post-Approval Flagging	18
15 Local and Global Database	19
16 E-Mail Integration	20
16.1 E-Mail Features	20
17 Browser Automation	21
18 Queue System	22
19 Privacy Score	23
20 Document Protection	24
21 Plugins	25
22 AI Policy	26
23 Chapter Title Here	27
24 Notifications	28
25 Backup System	29
25.1 Integrity Verification	29
26 Updates	30
27 Interface	31
28 Security Philosophy	32
29 Threat Model	33

30 Licensing	34
30.1 Decision Rationale	34
30.2 What This Means in Practice	34
31 Project Identity	35
31.1 Current Development Status	35
32 Development Roadmap	37
32.1 Phase 1 Finalize the Requirements Specification	37
32.2 Phase 2 System Architecture	37
32.3 Phase 3 Database and Data Model Design.....	38
32.4 Phase 4 Repository and Development Environment.....	38
32.5 Phase 5 Security Foundation	39
32.6 Phase 6 Core Application Framework.....	39
32.7 Phase 7 Identity and Privacy Management	40
32.8 Phase 8 Company Database System	40
32.9 Phase 9 Request Generation Engine.....	40
32.10 Phase 10 Email Integration.....	41
32.11 Phase 11 Automation Framework.....	41
32.12 Phase 12 Plugin System.....	42
32.13 Phase 13 User Experience Refinement	42
32.14 Phase 14 Testing and Securtiy Review	42
32.15 Phase 15 Public Release	43
32.16 Guiding Principle	43

Project Overview

This project is a privacy-focused, local-first software platform designed to help individuals exercise their privacy rights against companies, data brokers, marketing databases, and other organizations that store or process personal information.

Design Principle

Automation only assists the user, it never replaces the user's decisions.

This project aims to provide an alternative to centralized privacy removal services by ensuring that:

1.1 Goals

- The user owns and controls their data.
- No mandatory online account exists.
- No central database of user identities exists.
- Automation assists users rather than replacing their decisions
- companies receive only the minimum information required
- the community maintains and improves company information

The software is designed as a serious security and consumer privacy application.

Core Philosophy

2.1 Local-First Architecture

All personal user information remains on the user's own device. The software must not require:

- cloud storage
- external user accounts
- centralized identity databases
- mandatory telemetry
- tracking systems

Any future hosted version must preserve privacy by ensuring user data remains encrypted and inaccessible to service operators.

2.2 User-Controlled Automation

The software is an automation tool, not an independent decision maker. The user always controls:

- which companies are contacted
- which requests are sent
- which information is shared
- whether documents are uploaded
- whether actions are approved

The system should automate repetitive tasks, while keeping final authority with the user.

2.3 Priority Order

The project priorities are:

1. Legal Compliance
2. Privacy
3. Extensibility
4. Maximum automation

5. Transparency
6. Community maintenance
7. Speed

Privacy Principles

3.1 Data Minimization

The system must always send the minimum amount of information required.

Example:

If a company only requires an email address, the software should not send:

- phone number
- address
- date of birth
- government identification

3.2 No Central User Database

The project must never maintain a database containing:

- user identities
- personal information
- request histories
- private documents

All sensitive information belongs to the user.

Supported Platforms

Initial supported platforms are:

- Windows
- Linux
- macOS

In the future it is possible that an *Android/iOS* companion application will be developed.

The first priority is desktop functionality. In later stages of the project, a webserver hosted version could be possible, where no user data leaves the browser and is only processed within the users personal electronic device. To save and/or change anything, a database would be downloaded through said web-interface, giving maximum privacy to the user and ensuring no data is transmitted through networks.

Installation

Installation should be as simple as possible. Target experience:

1. Install application
2. Create encrypted local vault
3. Configure privacy preferences
4. Begin using software

The system should manage dependencies automatically where possible.

User Authentication and Encryption

6.1 Local Authentication

The initial version uses local authentication. No online login system exists. A system for future web hosted systems would need other forms of login methods to ensure maximum security, this will need to be developed if the project moves in the web-hosted direction. Please note that the web-version would only ever be an alternative, the local offline program will always stay the default.

6.2 Password Requirements

Passwords must:

- contain at least 12 characters
- contain at least one number
- contain at least one special character

Weak passwords should be rejected including:

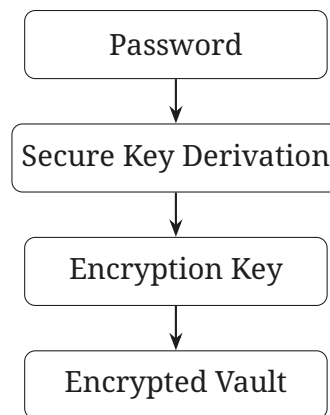
- common words
- predictable substitutions
- simple leetspeak modifications

6.3 Encryption Model

The software uses one encrypted vault. The vault contains:

- identities
- emails
- phone numbers
- addresses
- documents
- request history
- templates
- rules
- company preferences

The password is never stored directly:



6.4 Recovery

The vault uses a key-slot architecture, sometimes called "multi-key encryption" (the same underlying concept used by LUKS full-disk encryption and by emergency-access features in password managers such as Bitwarden). This section explains the mechanism in full, since it is the foundation of the project's security and recovery guarantees.

The Core Idea: Separating "What Encrypts the Data" from "What Unlocks the Vault"
A common mistake in vault design is to derive the encryption key directly from the user's password. If that were the case here, adding a second unlock method (like a recovery key) would require re-encrypting the entire vault with a new key every time. This slow, risky, and hard to do safely for large vaults. Instead, this project uses a two-layer model:

6.4.1 Layer 1 — The Data Encryption Key (DEK)

A single, randomly generated encryption key, created once when the vault is first set up. The DEK is what actually encrypts and decrypts the vault's contents (identities, emails, phone numbers, addresses, documents, request history, templates, rules, company preferences). The DEK itself is never stored in plaintext anywhere, and it is never derived from the user's password. It exists only in memory while the vault is unlocked.

6.4.2 Layer 2 — Key Slots

The DEK is never used directly by a human-facing credential. Instead, each credential that should be able to unlock the vault gets its own key slot: A key slot stores an encrypted copy of the DEK, wrapped (encrypted) using a key derived from that specific credential. "Wrapping" here means encrypting the DEK itself as a small piece of data, using standard key-wrapping practice (e.g., a KDF-derived key used with an authenticated encryption scheme). The wrapped DEK is small (a few dozen bytes) regardless of how large the vault is. A vault can have multiple key slots active at once, each independent of the others.

6.5 How unlocking works

Password Slot: When the user enters their password, it passes through secure key derivation, producing a key. That key attempts to unwrap the DEK stored in the password's key slot. If successful, the DEK is now available in memory, and the vault decrypts. **Recovery Key Slot:** The recovery key works identically, but through its own independent slot. The recovery key is not derived from the password, has no mathematical relationship to it, and does not need to be entered together with it. Either credential, used alone, fully unlocks the vault.

6.6 Why This Is Secure

Because password and recovery key each unlock their own independent slot, compromising one does not compromise or reveal the other. An attacker who steals the recovery key gains no information that helps them guess the password, and vice versa. The recovery key itself is generated using a cryptographically secure random number generator (CSPRNG), not a human-chosen phrase, meaning it carries full entropy and is not vulnerable to dictionary or pattern-based guessing the way passwords can be. The recovery key is displayed to the user exactly once, at the moment of vault creation (or regeneration — see below), and is never stored by the software in plaintext or in any recoverable form afterward. The user is responsible for storing it offline (e.g., printed on paper, written down, or kept in a physically separate secure location), consistent with the project's no-central-database and no-telemetry principles (Sections 2.1 and 3).

6.7 Recovery Key Regeneration

Because the DEK itself never changes when a key slot is added, removed, or replaced, the recovery key can be regenerated at any time without ever touching or re-encrypting the vault's actual contents. **The process:** The user must already be authenticated — either via their password or via a currently valid recovery key. A lost recovery key cannot be used to generate its own replacement; regeneration always requires proving access through a still-valid credential first. Once authenticated, the DEK is already available in memory (it was unwrapped during login). A new recovery key is generated fresh, using the same CSPRNG process as the original, with zero relationship to the previous key. The DEK is wrapped again using this new key, and stored in a new key slot. The old recovery key's slot is immediately and permanently destroyed. From that moment, the old recovery key is cryptographically useless — it wraps nothing, so even if it were later exposed, it grants no access. Because the DEK is unchanged throughout this process, regeneration is fast (a few dozen bytes rewritten) and does not scale with vault size, and it never touches the encrypted vault contents itself.

6.8 Why This Preserves the Security Guarantee

This design ensures that losing both the password and the (current) recovery key still results in fully unrecoverable data — there is no hidden master key or backdoor, and no third party, including the project itself, ever has access to the DEK or any key slot in plaintext. The recovery key is purely an additional, user-held, offline credential; it does not weaken the zero-knowledge nature of the vault.

6.9 Opt-Out

If a user declines to generate a recovery key at vault creation, the vault falls back to the original no-recovery behavior: a lost password means lost data, with no fallback of any kind. This remains available as an explicit opt-out for users who prioritize maximum security over convenience, and can be revisited later by generating a recovery key at any point after vault creation, following the same authenticated regeneration flow described above.

Identity Management

Users can create unlimited identity information. Supported information for 1.0:

- names
- email addresses
- phone numbers
- addresses
- previous addresses
- business identities
- historical identities

7.1 Identity Profiles

Users can organize information into profiles. Examples:

- Personal Identity
- Business Identity
- Previous Address
- Old Email Identity
- Custom Profiles

Companies only receive information from selected profiles

Privacy Rules

Users can create, manage, and share custom rules. Examples:

- Never automatically upload government ID
- Always request confirmation before sending documents
- Never send phone numbers unless required
- Always remove metadata from files
- Require approval for high-risk requests

The default rules are very strict. This ensures the user will have full security, and only information that the user opts-in to will be shared.

Legal Framework Support

The system should support major privacy frameworks worldwide. Included from version 1.0 should be:

- GDPR
- UK GDPR
- CCPA/CPRA
- US state privacy laws

The system should expand support over time. Near-term expansion targets, to be designed for from the start so the Phase 9 template engine does not require a later redesign, include:

- Canada (PIPEDA)
- Brazil (LGPD)
- Australia (Privacy Act)

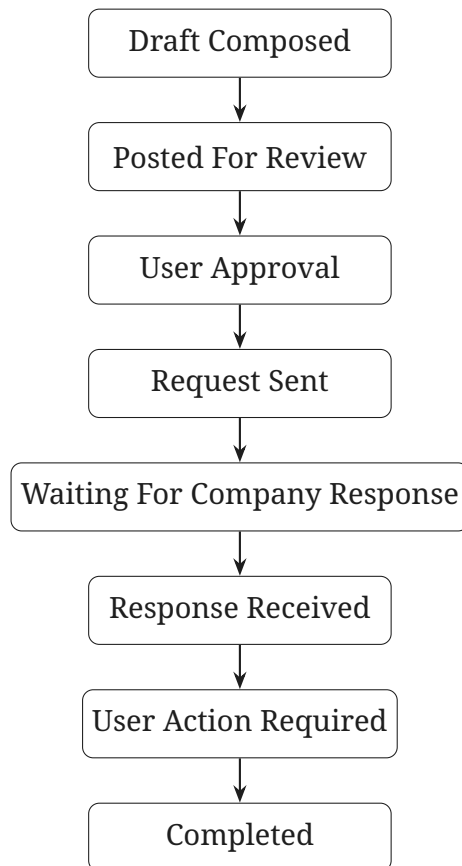
Privacy Request Types

Supported requests include (depending on the legal framework in use by the user):

- deletion
- access
- correction
- restriction
- portability
- objection
- opt-out

Request Workflow

Requests follow a clear lifecycle:



The system never performs sensitive actions without user approval.

Bulk Actions

The software supports bulk privacy actions. Examples:

- Remove me from all known people-search websites.
- Remove me from advertising databases.
- Remove me from data brokers.
- Bulk requests should be grouped intelligently.
- Similar requests may be approved together.
- Different requests should be separated.
- Users can create custom groups.

Company Database

The company database is a core component. It contains:

- company information
- privacy contacts
- legal requirements
- request methods
- supported frameworks
- automation capabilities

13.1 Company Records

Every company record must contain minimum information required for privacy requests. Required:

- company name
- website/domain
- privacy contact
- request method

Additional information:

- privacy URLs
- email addresses
- addresses
- supported laws
- identity requirements
- deadlines
- request history
- automation methods

Community Database System

The global company database is community maintained. Users can submit additions and corrections directly through the software. The software converts submissions into the correct database format.

14.1 Review Process

User submits information. Software validates the submission. Community votes. Reputation influences community voting. High-status changes require moderator approval. At least 70 Moderator votes are independent of reputation.

14.2 Submission Verification

Because company records can redirect real personal data (e.g., a privacy contact email), every submission — not only high-status changes — must pass at least one independent verification step before being usable, regardless of vote count or submitter reputation. Examples of independent verification include:

- confirming a submitted privacy contact address belongs to the same domain as the company's official website
- cross-checking a submitted request method against publicly published company privacy policy pages flagging submissions where contact details do not match previously known-good records for the same company.

Submissions that fail automated verification are held for manual moderator review before becoming usable, regardless of community vote outcome.

14.3 Post-Approval Flagging

Approval is not the end of a record's scrutiny. Any user can flag a live, already-approved company record as suspicious (e.g., a changed privacy contact, a request that behaved unexpectedly, a mismatch with the company's official site). Records that accumulate enough flags are automatically suspended from active use pending moderator review, regardless of their prior approval status or reputation-weighted vote history. This ensures that a record which was legitimate at approval time but is later compromised, or that passed review due to coordinated abuse, can still be caught and pulled out of circulation quickly.

Local and Global Database

Each installation has:

- **Global Database**
 - Community maintained
- **Local Database**
 - User controlled

The local database allows users to store companies that are not yet globally approved.

E-Mail Integration

The software connects to existing email providers.
Supported:

- Gmail
- Proton Mail
- Outlook
- generic IMAP/SMTP

Users should be encouraged to create a dedicated email account only for privacy requests and then link that account to the software. Users should not use their day-to-day email address.

16.1 E-Mail Features

The system should:

- send requests
- receive replies
- organize conversations
- track deadlines
- detect verification requests
- maintain request history

Browser Automation

Automation should be flexible.

Supported approaches:

- Full Automation
 - For simple predictable systems.
- Assisted Automation
 - The software fills forms while the user completes required steps.
- Manual Assistance
 - For complicated websites.
 - The system must never bypass security protections such as CAPTCHA.

The system must always respect a target site's Terms of Service and robots.txt directives. Automation is limited to legitimate, user-authorized interactions with a company's own privacy-request channels; it must never be used in a way that resembles unauthorized scraping, credential stuffing, or circumvention of access controls. This boundary protects both the user and the project's legal standing, and is core to the project's positioning as a serious, trustworthy security application (Section 31).

Queue System

Background Operations require task queue.

The preferred initial solution is an embedded, SQLite-backed queue.

This choice preserves the local-first and simple-installation principles established in Sections 2.1 and 5: no external database server needs to be installed, configured, or maintained by the user, and the queue lives inside the same local application footprint as the rest of the vault-adjacent data.

Used for:

- sending emails
- scheduled checks
- database synchronization
- document processing
- background tasks

This reduces dependencies and improves local security.

Privacy Score

Before requests are sent, the user should see a privacy impact score.

LOW RISK

- Email only

HIGH RISK

- Government ID required
- Manual approval required

The score informs the user but does not prevent action.

Document Protection

Before transmitting documents, the software should:

- remove metadata
- remove EXIF information
- remove hidden properties
- remove unnecessary content

Supported features:

- redaction
- cropping
- watermarking

Plugins

Plugins allow community expansion.

Examples:

- specialized company automation allowing more complicated requests to be automated
- optional AI tools, for example to categorize emails more effectively
- additional integrations

Plugins must:

- be open source
- be reviewed
- be signed
- be reproducible
- use permission controls

Default plugin permissions:

! No access.

Plugins must explicitly be granted access by the user.

AI Policy

AI is not part of the core application.
Possible AI features must be:

- optional
- local only
- open source
- permission controlled

No external AI communication is allowed. All AI integrations must stay within the program itself.

Chapter Title Here

Default logs contain no personal information.

Allowed:

"Request AZ-7492 submitted on 2026-05-23 at 1724Z."

Not allowed:

"Request submitted for user@email.com."

Users may enable separate personal logs for their own review. These logs would include PID and PII such as emails, companies, and names.

Notifications

Notifications are disabled by default.

Optional notification:

Email sent through the user's configured email system.

No third-party push notification systems.

Possibility to opt-in for a background service that sends push notifications. This is not yet part of the projects scope.

Backup System

Encrypted backups are supported. Backups may contain:

- settings
- vault
- hoistory

Backups should never expose unencrypted personal data.

25.1 Integrity Verification

Before any restore operation is performed, the backup file must be integrity-checked using a checksum and signature verification, confirming the file has not been corrupted or tampered with since it was created. A restore must be blocked, with a clear warning shown to the user, if integrity verification fails. This protects against both accidental corruption (e.g., an incomplete file copy) and malicious tampering (e.g., a modified backup file being reintroduced to the system).

Updates

Updates require prior user approval.

There are several security requirements for the update system:

- signed releases
- verified developers
- reproducible builds where possible

Everytime the software is launched, it checks for database and software updates, and prompts the user for their approval to install possible updates.

The community company database sync channel is held to the same signature verification standard as software updates. Every database update package must be signed by a verified source and cryptographically verified by the client before being applied. This closes the supply-chain gap that would otherwise exist if only software binaries were verified while a malicious or compromised database push could be applied unchecked (see Section 29, Threat Model).

Interface

The application uses one advanced interface. This interface consists of the following main sections:

- Requests
- Companies
- Identity Profiles
- Rules
- Plugins
- Logs
- Settings

A command-line interface (CLI) may be added later. This will likely be limited to UNIX-based operating systems.

Security Philosophy

Security should be community driven. The project encourages:

- code review
- vulnerability reports
- ethical security testing
- penetration testing

Users are explicitly encouraged to test the software responsibly to improve security.

Threat Model

The software protects against:

- centralized data collection
- cloud provider compromise
- stolen encrypted databases
- accidental information oversharing
- malicious plugins
- malicious or compromised database/software update pushes (see Section 26)

The software cannot protect against:

- malware already controlling the device
- companies legally requiring information
- users exposing their own passwords

Licensing

License Direction: Source-available, non-commercial (e.g. PolyForm-Noncommercial-1.0.0 or a comparable non-commercial license).

30.1 Decision Rationale

The project's two goals, full source transparency and a hard block on commercial exploitation, cannot both be satisfied by a license that meets the formal Open Source Definition (OSD). The OSD's "no discrimination against fields of endeavor" clause specifically prohibits restricting commercial use; this is the line that separates "open source" (OSI-recognized licenses such as MIT, Apache-2.0, GPL/AGPL) from "source-available" licenses (such as BUSL or PolyForm-Noncommercial). Because preventing commercial exploitation is a non-negotiable priority for this project (see Section 31), the project deliberately chooses a source-available, non-commercial license over an OSI-approved open-source license.

30.2 What This Means in Practice

The full source code is publicly available to read, audit, fork, and modify, with no restriction on personal, educational, research, or nonprofit use. Community contribution remains fully open. No individual, company, or derivative project may generate revenue from this codebase, or any part of it, without a separate agreement with the project. This license technically does not qualify as "open source" under the OSI's formal definition, and the project will not claim otherwise. It will be described accurately as "source-available" or "free and open, non-commercial," with this document's rationale made publicly visible so the distinction is never misrepresented to users or contributors. This decision is intentional and explained publicly, rather than left ambiguous, so that the project's transparency claims (Section 31) remain accurate and defensible.

The exact license text (e.g. PolyForm-Noncommercial-1.0.0 vs. a custom non-commercial license) will be finalized during Phase 1, but the direction, source-available, non-commercial, is now settled.

Project Identity

The project should be positioned as a serious security and consumer privacy application.

It should be:

- professional
- trustworthy
- transparent
- privacy-focused

It should not become:

- surveillance software
- a data collection service
- a marketing platform

And no exceptions can be made by any entity reusing the entire codebase, or parts thereof.

31.1 Current Development Status

Completed:

- project philosophy
- privacy principles
- user model
- company database concept
- automation approach
- security requirements
- community model

Not started:

- implementation
- architecture
- database schema
- code
- deployment system



Development Roadmap

The project will follow a structured, phase-based development process. No implementation should begin until the current specification document has been reviewed and approved. Each phase builds upon the previous one and should not begin until the preceding phase has reached a stable state.

32.1 Phase 1 Finalize the Requirements Specification

Objective: Establish a complete and unambiguous definition of the project.

Tasks:

- ☐ Review and refine this specification document.
- ☐ Resolve any remaining ambiguities or conflicting requirements
- ☐ Finalize the project's scope for Version 1.0.
- ☐ Decide on the project's license.
- ☐ Define contribution guidelines and coding standards.
- ☐ Freeze the specification before architectural work begins.

Deliverables:

- Approved Project Requirements Specification
- Contribution Guidelines
- Initial Governance Model

32.2 Phase 2 System Architecture

Objective: Design the overall architecture of the application before any code is written.

Tasks:

- ☐ Define the complete application architecture.
- ☐ Identify all major components and their responsibilities.
- ☐ Define communication between components.
- ☐ Design the plugin architecture.
- ☐ Define the security model.
- ☐ Define the permission model.
- ☐ Define the update mechanism.

- Produce a threat model identifying security assumptions and risks.

Deliverables:

- System Architecture Document
- Threat Model
- Component Diagram
- Data Flow Diagram

32.3 Phase 3 Database and Data Model Design

Objective: Design all persistent data structures.

Tasks:

- Design the SQLite database schema.
- Define the encrypted vault structure.
- Design company database import/export formats.
- Define the YAML schema for global company records.
- Design the local company database.
- Design identity profile storage.
- Design request history storage.
- Design rule storage.
- Design plugin metadata storage.

Deliverables:

- Database Schema
- YAML Specification
- Entity Relationship Diagram (ERD)

32.4 Phase 4 Repository and Development Environment

Objective: Prepare the project for collaborative development.

Tasks:

- Create the repository structure.
- Define branching strategy.
- Configure issue templates.
- Configure pull request templates.
- Configure automated formatting.
- Configure linting.
- Configure static analysis.

- ☐ Configure testing framework.
- ☐ Configure continuous integration.

Deliverables:

- Repository Structure
- Development Guidelines
- CI Pipeline

32.5 Phase 5 Security Foundation

Objective: Implement and verify the project's security foundation before any user-facing features.

Tasks:

- ☐ Implement encrypted vault architecture.
- ☐ Implement password handling.
- ☐ Implement encryption key derivation.
- ☐ Implement secure configuration storage.
- ☐ Implement secure logging.
- ☐ Implement permission framework.
- ☐ Implement plugin sandboxing framework.

Deliverables:

- Secure Vault
- Authentication Module
- Permission Framework

32.6 Phase 6 Core Application Framework

Objective: Build the application's core infrastructure.

Tasks:

- ☐ Implement the GUI framework.
- ☐ Implement application navigation.
- ☐ Implement settings management.
- ☐ Implement background task queue.
- ☐ Implement update checking.
- ☐ Implement configuration management.
- ☐ Implement backup framework.

Deliverables:

- Functional Application Shell

32.7 Phase 7 Identity and Privacy Management

Objective: Implement user-controlled identity management.

Tasks:

- ☐ Identity profile management.
- ☐ Contact information management.
- ☐ Privacy rule engine.
- ☐ Privacy score calculation.
- ☐ Data minimization engine.
- ☐ Document processing pipeline.
- ☐ Metadata removal.
- ☐ Redaction tools.

Deliverables:

- Identity Manager
- Privacy Rule Engine
- Privacy Processing Pipeline

32.8 Phase 8 Company Database System

Objective: Build the complete company database infrastructure.

Tasks:

- ☐ Import global company database.
- ☐ Implement local company database.
- ☐ Company search.
- ☐ Company categorization.
- ☐ Company submission interface.
- ☐ Community submission generation.
- ☐ Synchronization framework.
- ☐ Version history support.

Deliverables:

- Company Database Manager
- Synchronization System

32.9 Phase 9 Request Generation Engine

Objective: Generate legally compliant privacy requests.

Tasks:

- ☐ Implement legal request templates.
- ☐ Support multiple privacy frameworks.
- ☐ Support company-specific templates.
- ☐ User-editable templates.
- ☐ Company-specific template overrides.
- ☐ Bulk request generation.
- ☐ Request preview system.

Deliverables:

- Request Generation Engine

32.10 Phase 10 Email Integration

Objective: Enable communication with companies.

Tasks:

- ☐ SMTP support.
- ☐ IMAP support.
- ☐ Gmail integration.
- ☐ Proton Mail integration.
- ☐ Outlook integration.
- ☐ Generic mail provider support.
- ☐ Email tracking.
- ☐ Conversation management.
- ☐ Response parsing.
- ☐ Bounce detection.

Deliverables:

- Email Communication Module

32.11 Phase 11 Automation Framework

Objective: Automate interactions where appropriate.

Tasks:

- ☐ Email automation.
- ☐ Assisted browser automation.
- ☐ Manual workflow support.
- ☐ CAPTCHA pause handling.
- ☐ Automation plugin API.

- Automation status reporting.

Deliverables:

- Automation Framework

32.12 Phase 12 Plugin System

Objective: Allow safe community extensions.

Tasks:

- Plugin loading.
- Plugin verification.
- Digital signature verification.
- Sandboxing.
- Permission requests.
- Plugin management interface.

Deliverables:

- Plugin Framework

32.13 Phase 13 User Experience Refinement

Objective: Improve usability without compromising functionality.

Tasks:

- Improve workflows.
- Improve accessibility.
- Improve search.
- Improve navigation.
- Improve documentation.
- Improve error handling.
- Improve onboarding.

Deliverables:

- Feature-Complete User Interface

32.14 Phase 14 Testing and Security Review

Objective: Verify stability, correctness, and security.

Tasks:

- Unit testing.
- Integration testing.
- End-to-end testing.

- ☐ Security testing.
- ☐ Fuzz testing where appropriate.
- ☐ Performance testing.
- ☐ Community security review.
- ☐ Resolve reported issues.

Deliverables:

- Release Candidate

32.15 Phase 15 Public Release

Objective: Publish Version 1.0.

Tasks:

- ☐ Publish documentation.
- ☐ Publish installation instructions.
- ☐ Publish contribution guidelines.
- ☐ Publish security policy.
- ☐ Publish roadmap.
- ☐ Publish release binaries.
- ☐ Open community contributions.

Deliverables:

- Version 1.0 Release

32.16 Guiding Principle

Development should always prioritize correctness, security, and privacy over rapid feature delivery. Features should only be implemented once the underlying architecture and security model supporting them have been fully designed and reviewed. At every stage of development, maintaining user privacy and preserving local control over sensitive information take precedence over convenience or development speed.